

Simulation Object Versioning Module (SOVM)

OOF™ Origin Open Foundation™

Independent Methodological Authority

Architecture Ecosystem: Structured Reality Standards™

Architecture Family: SIMULOS® — Simulation Governance Architecture

Parent Standard: Simulation Object Standard (SOS)

Operational Layer: Simulation Object Governance Layer

Category: Governance & Enforcement

Subcategory: Simulation Governance

Type: Parent Standard Module

Governed Space: Simulation Object Versioning

Version: 1.0

Status: Canonical · Open Module

Effective Date: 15 August 2026

Compatibility: OOF® Methodology OS™ · GOA™ · ORA™ · VALIDOS® · INTEGROS® · ADIT® · AGA™ · AIG® · CLIA® · MGIA™ · ASGA™ · RIS™

AI-Readable: Yes

Authority: OOF®

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Governed Space

Simulation Object Versioning

Canonical Definition

Simulation Object Versioning Module (SOVM) defines the governance conditions under which materially distinguishable versions of a Simulation Object are identified, differentiated, related, transitioned, and maintained as traceable simulation-governance objects throughout the simulation lifecycle.

SOVM ensures that simulation activity, evidence, outputs, findings, and downstream reliance remain bound to the version of the Simulation Object to which they actually apply.

Operational Role

SOVM serves as the third internal governance space of the Simulation Object Standard (SOS).

The preceding modules establish:

SOIM — What exactly is the Simulation Object?

SOBM — Where does the Simulation Object begin and end?

SOVM asks:

Which version of that Simulation Object is being simulated, and when does change create a materially different version?

The governed progression is:

Identified & Bounded Simulation Object → Version Basis → Version Identity →
Material Change Assessment → Version Relationship → Version Status → Version
Traceability → Governed Simulation Object Version

SOVM prevents simulation conclusions established for one object version from silently migrating to another.

Module Operational Space

SOVM governs: Simulation Object version identity, version designation, version basis, version differentiation, version chronology, version relationships, predecessor versions, successor versions, parallel versions, derived versions, version baselines, version transitions, material version change, minor change, major change, version ambiguity, version conflict, version equivalence, version applicability, version status, version lineage, version continuity, version supersession, version retirement, historical version preservation, version-to-simulation binding, version-to-output binding, cross-version applicability, and version traceability.

Module Function

SOVM exists because identifying a Simulation Object once is insufficient.

Objects change.

Software is updated.

Hardware is modified.

AI models evolve.

Control logic changes.

Processes are redesigned.

Components are replaced.

Operational capabilities expand or contract.

A simulation result produced against an earlier version may therefore cease to represent the object that exists later.

SOVM prevents: unknown object versions, version ambiguity, silent version substitution, uncontrolled inheritance of simulation findings, simulation outputs being detached from their object version, materially changed objects being represented as unchanged, different versions being aggregated without disclosure, superseded versions being confused with current versions, and historical simulation conclusions being represented as current without a

governed basis.

Simulation Object Version

A Simulation Object Version is a distinguishable state of a Simulation Object defined by a set of characteristics sufficient to determine which object state was represented by a simulation.

Version identity may depend upon: design revision, software release, model release, hardware revision, structural modification, capability change, process revision, control-logic change, operational change, or another materially relevant object change.

A version does not require a particular commercial or technical versioning convention.

SOVM governs methodological distinguishability.

Version Basis

A Version Basis establishes which characteristics determine whether two states of a Simulation Object should be treated as:

the same version

or:

different versions

for simulation-governance purposes.

The Version Basis should reflect characteristics capable of materially changing: simulated behavior, simulation assumptions, applicable scenarios, interactions, expected outputs, reality correlation, or interpretation of simulation results.

Technical Version ≠ Governed Simulation Version

SIMULOS® establishes:

Technical Version Identity ≠ Simulation Governance Version Identity

A vendor may label two builds:

v3.1.1

and:

v3.1.2

while the difference is immaterial to a particular simulation.

Conversely, an operational change may materially alter simulation relevance

even where no formal software version number changes.

SOVM therefore does not depend exclusively upon external version labels.

Version Identification

Every simulation should be sufficiently bound to an identifiable Simulation Object Version where version differences could materially affect the simulation.

Version identification may include: version identifier, release identifier, revision identifier, build identifier, object state, effective date, material characteristics, predecessor relationship, or other distinguishing information.

Material Version Change

A Material Version Change is a change capable of affecting the validity, applicability, interpretation, or transferability of simulation activity or simulation-derived results.

Possible examples include: changed system behavior, changed physical characteristics, changed algorithms, changed decision logic, changed interfaces, changed capabilities, changed autonomy, changed safety mechanisms, changed operating modes, or changed material dependencies.

A material change should create either:

a new governed Simulation Object Version

or:

an explicit determination explaining why version continuity remains justified.

Minor Change

A change may be classified as minor where it does not materially affect the simulation-relevant characteristics of the Simulation Object.

Minor changes should not automatically require creation of a new governed simulation version.

However, the basis for treating a potentially relevant change as minor should remain reconstructable.

Major Change

A major change materially alters one or more characteristics relevant to simulation behavior or interpretation.

A major change normally requires a distinguishable Simulation Object Version.

Version Significance Is Simulation-Relative

The significance of a change may depend upon the simulation purpose.

A modification irrelevant to one simulation may be material to another.

For example:

a cosmetic physical change may be irrelevant to a software-behavior simulation,

but material to an aerodynamic simulation.

Therefore:

Version Materiality must be evaluated relative to the governed simulation context.

Version Baseline

A Simulation Object Version Baseline establishes the reference version against which relevant changes are evaluated.

The baseline may identify: object identity, boundary, version identifier, material characteristics, effective state, known limitations, and applicable simulation relationship.

Version Transition

A Version Transition occurs when the Simulation Object moves from one governed version to another.

Conceptually:

Version A → Material Change → Version B

The transition should preserve the relationship between predecessor and successor.

Version Lineage

Version Lineage establishes how Simulation Object Versions relate across time.

For example:

V1 → V2 → V3

or:

V1 → V2A

V1 → V2B

where parallel development produces divergent versions.

Version lineage allows simulation evidence and findings to remain attached to the correct historical object state.

Parallel Versions

Multiple versions of the same Simulation Object may exist simultaneously.

For example: production version, experimental version, regional version, safety-modified version, prototype version, or alternative design version.

SOVM requires each materially distinguishable version to remain identifiable.

Derived Version

A Derived Version originates from an existing Simulation Object Version but contains changes sufficient to require independent version identity.

Its relationship to the originating version should remain traceable.

Version-to-Simulation Binding

Every governed simulation should be capable of answering:

Which Simulation Object Version did this simulation represent?

This creates the relationship:

Simulation Object → Object Version → Simulation Activity

Without this binding, later interpretation may incorrectly apply simulation findings to another version.

Version-to-Output Binding

Simulation outputs should remain traceable to the Simulation Object Version from which they were generated.

Conceptually:

Object Version → Simulation Execution → Simulation Output

SOVM governs the version relationship.

It does not govern the substantive output itself, which belongs to the relevant downstream SIMULOS® governance space.

No Silent Version Substitution Rule

SIMULOS® establishes:

A Simulation Object Version must not be replaced by another materially different version without explicit governance of the change.

A simulation started against Version A must not silently become represented as

a simulation of Version B.

No Automatic Result Inheritance Rule

SIMULOS® establishes:

Simulation results established for one Simulation Object Version do not automatically apply to another version.

Cross-version applicability requires a governed basis.

Cross-Version Applicability

A simulation result from Version A may remain relevant to Version B where the changed characteristics do not materially affect the simulation question.

SOVM permits such applicability, but requires the relationship to be explicit.

Possible statuses may include:

Fully Applicable

Conditionally Applicable

Partially Applicable

Not Applicable

or:

Applicability Undetermined Version Equivalence

Two formally different versions may be simulation-equivalent for a specific simulation purpose.

Version equivalence should not be assumed merely because the versions are similar.

It should be based upon the characteristics relevant to the simulation.

Version Divergence

Version Divergence exists where two versions become sufficiently different that simulation findings can no longer be safely treated as mutually applicable.

Divergence may result from: capability expansion, architecture change, behavior change, interface change, physical redesign, changed autonomy, or accumulated modifications.

Version Ambiguity

Version Ambiguity exists where the exact Simulation Object Version represented by a simulation cannot be established with sufficient confidence.

Material version ambiguity should remain visible.

It must not be silently converted into a specific version assignment.

Version Conflict

A Version Conflict exists where credible records identify materially different versions as the object represented by the same simulation activity.

The conflict should be resolved or explicitly preserved before downstream conclusions depend upon version identity.

Version Supersession

A newer Simulation Object Version may supersede an earlier version.

Supersession does not erase the earlier version.

Historical simulation records must continue to identify the version to which they originally applied.

Historical Version Preservation

SOVM preserves historical Simulation Object Versions because later questions may require reconstruction of: what object existed, which version was simulated, which findings applied, what changed afterward, and whether later reliance crossed a version boundary.

Version Retirement

A Simulation Object Version may be retired when it is no longer active or relevant for current operation.

Retirement does not delete: version identity, simulation history, outputs, lineage, or previous applicability.

Version Status

A Simulation Object Version may hold statuses such as:

Current

Active

Experimental

Parallel

Superseded

Retired

Ambiguous

or:

Undetermined

Status identifies the lifecycle position of the version.

It does not determine simulation validity.

Version Change Propagation

A material version change may affect downstream SIMULOS® governance, including: reality representation, assumptions, model and environment, inputs and data, scenario design, execution, outputs, reality correlation, simulation evidence, or operational reliance.

SOVM identifies and propagates the version change.

It does not replace the downstream governance spaces responsible for determining the consequences.

Version Change vs Configuration Change

SOVM distinguishes:

Version Change

from:

Configuration Change

A version identifies a materially distinguishable state or generation of the Simulation Object.

A configuration identifies the specific arrangement of relevant components, settings, parameters, options, dependencies, or operational features associated with that object.

A configuration change does not necessarily create a new version.

A version change may contain multiple configurations.

Detailed configuration governance belongs to SOCM — Simulation Object Configuration Module.

Version Change vs Boundary Change

A version change does not necessarily change the Simulation Object Boundary.

Likewise, a boundary change may be material enough to require a new version.

SOBM governs the boundary.

SOVM governs version identity and continuity.

Where both change, the relationship should remain traceable.

Version Change vs Validation

SOVM does not determine whether a changed object remains valid for operational reliance.

That question belongs to VALIDOS® where formal validation or revalidation is required.

SOVM provides the version-change basis that VALIDOS® may consume.

Version Change vs Integrity

SOVM does not determine whether the object has preserved integrity through change.

That governance belongs to INTEGROS®.

SOVM determines whether the change creates or affects a distinguishable Simulation Object Version.

Simulation Object Version Record

SOVM should establish a persistent Simulation Object Version Record sufficient to reconstruct which object version existed and which simulations were bound to it.

The record may preserve: Simulation Object Identifier, Simulation Object Version Identifier, Version Basis, version designation, effective period, material characteristics, predecessor version, successor version, parallel versions, derived-version relationships, material changes, minor changes, version status, version ambiguity, version conflicts, cross-version applicability, version lineage, supersession, retirement, simulation bindings, output references, and version traceability.

Failure Conditions

SOVM governance failure may exist where: a materially relevant Simulation Object Version is unidentified, simulations cannot be bound to the object version represented, materially different versions are treated as equivalent without basis, version changes occur silently, historical simulation results are automatically transferred to successor versions, version ambiguity is hidden, conflicting version records are ignored, superseded versions are represented as current, version lineage cannot be reconstructed, version changes are confused with configuration changes, or downstream simulation conclusions cannot be traced to the version from which they originated.

Minimum Implementation Framework

1. Establish the Version Basis

Define which characteristics are relevant for distinguishing Simulation Object Versions within the governed simulation context. 2. Identify the Simulation

Object Version

Assign or preserve sufficient version identity and bind it to the identified and bounded Simulation Object. 3. Assess Material Change

Determine whether relevant changes preserve the current version, create a new version, or leave version status unresolved. 4. Establish Version Relationships

Preserve predecessor, successor, parallel, derived, superseded, and applicable cross-version relationships where relevant. 5. Preserve Version Traceability

Bind simulation activity and resulting outputs to the applicable Simulation Object Version and maintain the historical version lineage.

Governance Outputs

SOVM may produce: Simulation Object Version Record, Simulation Object Version Identifier, Version Basis Record, Version Baseline, Version Materiality Assessment, Minor Change Record, Material Version Change Record, Version Transition Record, Version Relationship Map, Version Lineage Record, Predecessor Version Reference, Successor Version Reference, Parallel Version Record, Derived Version Record, Version Equivalence Assessment, Cross-Version Applicability Assessment, Version Divergence Record, Version Ambiguity Record, Version Conflict Record, Version Status, Version Supersession Record, Version Retirement Record, Version-to-Simulation Binding, Version-to-Output Reference, and Simulation Object Version Traceability Map.

Use Case 1 — Autonomous Vehicle Software Upgrade

Scenario

An autonomous vehicle has been simulated using:

Autonomous Driving System Version 7.2

A later release changes perception and decision logic.

The manufacturer labels the successor:

Version 7.3 Application

SOVM assesses whether the changes affect characteristics material to the simulations previously performed against Version 7.2.

Because perception and decision logic materially affect simulated driving behavior, Version 7.3 is maintained as a distinguishable Simulation Object Version.

Existing Version 7.2 simulation results remain historically valid records but do not automatically transfer to Version 7.3. Result

The relationship remains explicit:

Version 7.2 → Material Change → Version 7.3

Downstream SIMULOS® governance can determine which simulation activities must be repeated or reassessed. Use Case 2 — Spacecraft Hardware Revision

Scenario

A spacecraft component undergoes a hardware revision before launch.

The change affects thermal behavior but does not affect navigation software.
Application

SOVM records the new Simulation Object Version and identifies the thermal modification as the material change.

Previous navigation simulations may remain applicable where version equivalence is sufficiently established for that simulation purpose.

Previous thermal simulations are not automatically inherited. Result

SOVM prevents an all-or-nothing interpretation.

The new spacecraft version remains historically connected to the previous version while simulation applicability is bounded according to the characteristics actually affected by the change.

Architectural Position

Simulation Object Versioning Module (SOVM) is the third internal governance space of the Simulation Object Standard (SOS).

The internal progression is:

Simulation Object Identification → Simulation Object Boundary → Simulation Object Versioning

SOIM establishes:

What is being simulated?

SOBM establishes:

Where does that object begin and end?

SOVM establishes:

Which materially distinguishable version of that object is being simulated?

The next internal governance space, SOCM — Simulation Object Configuration Module, governs the specific configuration of that version.

SOVM does not replace configuration governance, reality-representation governance, integrity governance, validation governance, or operational-reality governance.

It governs specifically:

the identity, differentiation, continuity, lineage, and simulation applicability of Simulation Object Versions.

Foundational Principle

A simulation result belongs first to the Simulation Object Version that was actually simulated; applicability to another version must be governed, not assumed.

Canonical Closing Statement

Simulation Object Versioning Module (SOVM) establishes the version-governance layer of the Simulation Object by ensuring that materially distinguishable object states remain identified, differentiated, historically related, and traceable throughout the simulation lifecycle. Through version baselines, material-change assessment, version transitions, lineage, cross-version applicability, equivalence, divergence, supersession, historical preservation, and explicit simulation binding, SOVM prevents simulation findings established for one object version from silently becoming findings about another. By preserving the relationship between Simulation Object identity, object change, version history, simulation activity, and downstream outputs, SOVM provides the third foundational object-governance mechanism of SIMULOS®.

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>